NETWORK TRAFFIC ANALYSIS

for Cybersecurity Monitoring and Threat Detection

BLUE TEAM / NETWORK SECURITY PROJECT REPORT

Project Type	Blue Team / Network Security / Traffic Analysis
Primary Tools	Wireshark, Python 3, Scapy
Analysis Capture	captures/normal-traffic.pcapng
Capture Size	5,640 packets
Author	Dumpeti Sathwik
Programme	B.Tech Computer Science and Engineering
Specialization	Cyber Security & Blockchain Technology

This report documents a practical introductory Blue Team workflow for capturing, filtering, analyzing and documenting network traffic. The emphasis is on evidence-based analysis and avoiding false positives when interpreting normal workstation activity.

1. Executive Summary

Network traffic analysis is a fundamental activity in defensive cybersecurity because network communications provide evidence about hosts, services, protocols and connection patterns. This project uses a Wireshark packet capture and a Python/Scapy analyzer to build a baseline of ordinary workstation traffic and identify events that deserve further investigation.

The baseline capture contains 5,640 packets. The analyzed traffic is dominated by UDP and TCP, with ARP activity representing local network address resolution and a smaller set of packets classified as Other/Unclassified. The classification is intentionally conservative: a TCP reset, a failed DNS lookup, or a connection to a common service port is not treated as malicious without supporting evidence.

> **Important correction:** this report does not use the earlier incorrect statement of 14,005 occurrences on TCP destination port 443. That statistic has been removed. Port 443 is discussed qualitatively as the standard HTTPS service port where relevant, without presenting the obsolete occurrence count.

2. Objectives

• Capture and inspect network traffic in an authorized environment.

• Understand DNS, TCP, UDP, ARP and TLS-related traffic.

• Use Wireshark display filters to isolate protocol-specific evidence.

• Establish a normal traffic baseline before investigating anomalies.

• Use Python and Scapy to summarize packets, protocols, IP addresses and destination ports.

• Identify investigation points without incorrectly labeling ordinary traffic as malicious.

• Document observations and defensive recommendations in a repeatable format.

3. Tools and Environment

Tool / Platform	Purpose
Wireshark	Packet capture, protocol dissection and display filtering
Python 3 + Scapy	Automated packet and port statistics
Git / GitHub	Version control and project presentation
Windows 11	Analysis workstation

4. Methodology

The project follows a simple SOC/Blue Team investigation sequence. First, traffic is captured from an authorized workstation environment. The capture is then reviewed in Wireshark using protocol and flag filters. Finally, a Python analyzer provides a repeatable statistical summary that can be compared with the manual Wireshark observations.

Workflow

Project Objective → Lab Setup → Traffic Capture → Protocol Analysis → Filtering → Normal Baseline → Suspicious-Pattern Investigation → Evidence & Findings → Recommendations → Final Report

5. Capture Overview

Field	Value
Capture file	captures/normal-traffic.pcapng
Total packets	5,640
Primary local host	192.168.1.10
Local DNS/router observed	192.168.1.1
Analysis approach	Wireshark + Python/Scapy
Assessment type	Normal-traffic baseline with investigation points

6. Protocol Distribution

Protocol Category	Packets	Interpretation
UDP	3,233	Common for DNS, discovery and other application/network services
TCP	2,080	Connection-oriented application traffic including web/TLS sessions
ARP	237	Local IP-to-MAC resolution and related LAN activity
Other/Unclassified	90	Packets not mapped to the primary categories by the analyzer
Total	5,640	Complete packet count represented by the categories above

7. Wireshark Analysis

7.1 DNS Analysis

The DNS filter was used to inspect name-resolution activity. Requests were observed between the workstation and the local DNS/router address. Microsoft-related connectivity and service lookups are consistent with ordinary operating-system and application activity. Some queries returned 'No such name'. A failed lookup alone is not evidence of compromise; the domain, frequency, process context and related network activity should be considered before escalation.

```
Filter: dns
```

7.2 TCP Analysis

TCP traffic includes connection establishment, application sessions, HTTPS communication and connection termination/reset behavior. TCP SYN packets are useful for identifying connection attempts, while TCP RST packets can occur when a service is unavailable, a connection is rejected, or an endpoint terminates a session unexpectedly.

```
Filters: tcp and tcp.flags.syn == 1

TCP reset filter: tcp.flags.reset == 1
```

> The presence of RST packets is therefore treated as an investigation point rather than proof of an attack. Destination port 443 is associated with HTTPS/TLS service traffic in this capture. The obsolete '443 – 14,005 occurrences' statistic has deliberately been removed from this report.

7.3 ARP Analysis

ARP packets include requests, replies and probes associated with local network operation. ARP is expected on an Ethernet LAN and should be interpreted in context. A stronger indication of possible ARP spoofing would require evidence such as conflicting IP-to-MAC mappings, unexpected gateway MAC changes, repeated unsolicited ARP replies, or corroborating endpoint/network alerts.

```
Filter: arp
```

7.4 TLS Analysis

TLS filtering can be used to identify encrypted session establishment and related TLS records. Because the application payload is encrypted, packet analysis can often reveal metadata such as endpoints, timing, packet sizes and handshake information rather than the actual application content.

```
Filter: tls
```

8. Suspicious-Traffic Assessment

The purpose of this section is not to declare normal traffic malicious. Instead, it records patterns that a Blue Team analyst would investigate further and explains what additional evidence would be needed for a confident classification.

Indicator	Current interpretation	Next investigation step
Repeated TCP RST	Needs investigation; not inherently malicious	Check peer, timing, service state and application logs
Multiple HTTPS connections	Expected on modern workstations	Review destinations, process context and frequency
Failed DNS queries	Not malicious by itself	Check queried names, repetition and endpoint process
ARP requests/probes	Normal LAN behavior is likely	Check IP/MAC consistency and gateway mapping
Many connection attempts	Potential reconnaissance indicator if abnormal	Compare with baseline and inspect port/host distribution

9. Python Packet Analyzer

The project includes `scripts/packet_analyzer.py`. The script reads the capture with Scapy and calculates total packet count, protocol distribution, source IP counts, destination IP counts, and TCP/UDP destination-port statistics. This provides a repeatable first-pass summary before detailed packet inspection in Wireshark.

Representative summary:

```
Total packets captured: 5640
UDP: 3233
TCP: 2080
ARP: 237
Other/Unclassified: 90
```

> The protocol categories reconcile to the total capture count. Destination-port values should be reported from the analyzer output rather than copied from an older report version, which prevents stale statistics from being carried into the final documentation.

10. Evidence and Documentation

The project should retain screenshots that demonstrate the actual analysis performed. Recommended evidence includes the complete Wireshark capture view, DNS-filtered traffic, TCP traffic, TCP SYN packets, TCP RST packets, ARP traffic, TLS traffic and the Python analyzer output.

Evidence item	Purpose
Overall capture	Shows capture scope and packet volume
DNS filter	Demonstrates name-resolution analysis
TCP / SYN filter	Demonstrates connection analysis
RST filter	Documents connection-reset investigation points
ARP filter	Documents local address-resolution behavior
TLS filter	Documents encrypted-session analysis
Python output	Provides reproducible statistical evidence

11. Security Recommendations

• Maintain a normal traffic baseline for the workstation or lab network.

• Investigate repeated connection attempts, especially when many ports or hosts are targeted in a short period.

• Review DNS requests for unusual domains, high-frequency lookups or suspicious naming patterns.

• Monitor ARP activity for unexpected IP-to-MAC changes or gateway inconsistencies.

• Correlate packet evidence with endpoint, firewall, DNS and application logs.

• Restrict unnecessary services and outbound traffic using appropriate firewall controls.

• Preserve relevant packet captures and screenshots when an incident requires investigation.

• Avoid classifying traffic as malicious from a single packet-level indicator.

12. Limitations

This project analyzes a baseline workstation capture rather than a confirmed incident. Modern operating systems and applications generate substantial background traffic, including DNS requests, HTTPS connections, broadcast/multicast packets and connection resets. Encrypted traffic also limits visibility into application payloads. Consequently, the findings should be interpreted as a baseline and investigation exercise rather than a malware-detection verdict.

A future controlled laboratory capture can strengthen the project by comparing normal traffic with an explicitly authorized reconnaissance or repeated-connection scenario. Any active testing should be restricted to systems owned by the student or systems for which explicit authorization has been obtained.

13. Future Enhancements

• Add automated anomaly scoring based on baseline traffic.

• Generate CSV or HTML reports from packet statistics.

• Add protocol and IP visualizations.

• Integrate Suricata or Snort alerts for signature-based detection.

• Compare normal and controlled suspicious PCAP files automatically.

• Add a small SOC-style dashboard for traffic trends and investigation status.

• Correlate network findings with host-based security logs.

14. Conclusion

The project demonstrates an introductory but practical Blue Team workflow: capture traffic, identify protocols, apply Wireshark filters, summarize traffic programmatically, document evidence, and investigate unusual patterns without creating unsupported conclusions. The approach is suitable as a foundation for further SOC, network monitoring and incident-response exercises.

> The corrected report intentionally removes the obsolete '443 – 14,005 occurrences' value and does not reference a nonexistent `lab-traffic.pcapng` file. The documented baseline capture is `normal-traffic.pcapng`.

15. Author

Dumpeti Sathwik

B.Tech Computer Science and Engineering (Cyber Security & Blockchain Technology)

Project Type: Blue Team / Network Security / Traffic Analysis